EEMS

Backend Development Blueprint

ASP.NET Core | Modular architecture | API contracts | Security and workflows

IMPLEMENTATION-READY DESIGN BASELINE | VERSION 1.0

Audience: Solution architects, engineering leads, developers, QA, DevOps and project owners.

Scope: Multi-tenant election operations across parliamentary, upazila, union, municipality and city corporation elections.

Boundary: Administrative operations, compliance, personnel, budgets, center readiness, incident records and source-attributed results. No voter persuasion targeting or political-preference profiling.

Document control: Architecture baseline; adapt to current election law and obtain independent legal/security review before production.

Contents and delivery contract

Each numbered section is a development handoff. Deliver features with database migration, backend authorization, frontend permission guard, audit, tests and release notes. No section is production-complete until acceptance tests pass.

No.	Section
1	Module boundaries and solution layout
2	Authentication, tenant isolation and authorization
3	Module business rules and domain aggregates
4	API contract conventions
5	Detailed endpoint inventory
6	Application layer patterns and sample code
7	Approval and event workflows
8	Background processing and integrations
9	Security controls and operational observability
10	Testing, CI/CD and deployment
11	System invariants
12	Cross-team contracts

1. Module boundaries and solution layout

Use ASP.NET Core Web API modular monolith with domain/application/infrastructure/presentation boundaries. EF Core with Npgsql; Redis for caching; OpenTelemetry; background worker for exports and notifications.

```
src/EEMS.Api/
src/EEMS.SharedKernel/
src/EEMS.Modules.{Tenants,Identity,Geography,Elections,Candidates,
  Polling,Teams,Events,Finance,Inventory,Operations,Results,Reporting}/
tests/{Unit,Integration,Security,Architecture,Contract}/
deploy/{docker,nginx,monitoring}/
```

2. Authentication, tenant isolation and authorization

• Use a vetted OIDC provider, authorization code with PKCE for browser, secure HttpOnly cookie/BFF pattern where feasible; MFA for privileged roles.

• Validate issuer, audience, expiry, signature and tenant membership; tenant_id must come from server-verified identity/context.

• Permission policy + resource-based scope check: tenant, election, area, center and action; no admin bypass by route name.

• PostgreSQL RLS as defense in depth; SET LOCAL app.tenant_id inside each transaction; pooled connections must not leak context.

• Maker/checker must be distinct user identities for protected workflows; reject self-approval.

• All authorization failures should be audited without storing secrets.

3. Module business rules and domain aggregates

Aggregate	Core invariants
Tenant	Unique code, status controls access, subscription feature limits
Election	Election type, area, date, lifecycle; publish requires valid configuration
CandidateElection	Position belongs to election type; area within scope; unique active assignment
PollingCenter	Canonical center vs election-specific center assignment; verified status
Team	Member active in tenant; assignment bounded by election/area
Expense	Positive amount, budget category, invoice reference, immutable approved history
Incident	Time/source/center, review status, evidence integrity
ResultSubmission	Candidate counts nonnegative; source and verifier independent; official results separate

4. API contract conventions

• Prefix /api/v1; use REST resource nouns; pagination page/pageSize or cursor, max page size 100.

• All responses carry request correlation ID; validation errors use RFC 7807 ProblemDetails.

• Use Idempotency-Key on create/payment-like commands; ETag/rowversion for updates.

• Use 201 + Location for creates; 400 invalid input, 401 unauthenticated, 403 forbidden, 404 inaccessible/missing, 409 conflict, 422 domain validation.

• OpenAPI schema published in CI and versioned; backward-compatible field additions within v1.

```
POST /api/v1/elections
{ "electionName":"Example", "electionTypeId":1,
  "areaId":1002, "electionDate":"2027-01-10" }
201 Created
{ "electionId":"uuid", "status":"DRAFT", "version":1 }

422 application/problem+json
{ "type":".../validation", "title":"Invalid election area",
  "status":422, "traceId":"..." }
```

5. Detailed endpoint inventory

Module	Endpoints
Tenant	GET /tenants/current; POST /tenants; PATCH /tenants/{id}/status
Identity	GET /users; POST /users/invitations; GET /roles; PUT /roles/{id}/permissions; GET /navigation/my-menu
Geography	GET /areas/tree; GET /constituencies; POST /areas/imports
Election	GET/POST /elections; GET/PUT /elections/{id}; POST /elections/{id}/publish
Candidate	GET/POST /candidates; POST /candidate-elections; GET /candidate-elections/{id}
Polling	GET/POST /polling-centers; POST /elections/{id}/centers; POST /agent-assignments
Team	GET/POST /teams; POST /team-members; POST /tasks; PATCH /tasks/{id}
Events	GET/POST /events; POST /events/{id}/documents; POST /events/{id}/close
Finance	GET/POST /budgets; GET/POST /expenses; POST /expenses/{id}/submit; POST /expenses/{id}/approve
Inventory	GET/POST /items; POST /stock-movements; POST /asset-allocations
Operations	GET /readiness; POST /incidents; POST /incidents/{id}/review
Results	POST /results/submissions; POST /results/{id}/verify; GET /results/official
Reports	GET /reports/overview; GET /reports/expenses; GET /audit-logs

6. Application layer patterns and sample code

```
public sealed record CreateElectionCommand(
  string Name, short ElectionTypeId, long AreaId, DateOnly Date);

public async Task<Guid> Handle(CreateElectionCommand cmd,
    VerifiedTenantContext context, CancellationToken ct) {
  await authorization.RequireAsync(context, "election.create", ct);
  await rules.ValidateElectionAreaAsync(cmd.ElectionTypeId,cmd.AreaId,ct);
  var election = Election.Create(context.TenantId, cmd.Name,
    cmd.ElectionTypeId, cmd.AreaId, cmd.Date);
  db.Elections.Add(election);
  await db.SaveChangesAsync(ct);
  return election.Id;
}
```

• Example illustrates domain boundary; production adds transaction-scoped RLS, audit outbox, idempotency and exception mapping.

• Use FluentValidation or equivalent request validation, explicit domain errors, and EF Core configurations per aggregate.

• For critical commands, write domain data and outbox message atomically in same database transaction.

7. Approval and event workflows

Workflow	States	Guard
Election	DRAFT > ACTIVE > CLOSED	configuration complete; publish permission
Expense	DRAFT > SUBMITTED > REVIEW > APPROVED/RETURNED	maker/checker distinct; budget constraints
Incident	REPORTED > TRIAGED > ASSIGNED > RESOLVED > CLOSED	review evidence and resolution
Result	DRAFT > SUBMITTED > REVIEW > VERIFIED/DISPUTED > ARCHIVED	source evidence and independent verification
Inventory	RECEIVED > ALLOCATED > RETURNED/CONSUMED	balanced movement ledger

8. Background processing and integrations

• Hangfire or equivalent for report exports, document scanning, notification dispatch, reconciliation and archival.

• Outbox table for reliable event publishing; retry with exponential backoff and dead-letter handling.

• External election commission data import only through authorized sources; preserve provenance and timestamp.

• Notification templates localized; consent and frequency controls; no targeted voter persuasion messaging.

• Object storage with content hash, retention and signed URL expiry.

9. Security controls and operational observability

• Secrets in vault/environment secret provider; rotate credentials and signing keys.

• OWASP ASVS-inspired controls: rate limiting, request size limits, CSRF protections for cookie auth, SQL parameterization, safe uploads.

• Structured logs include tenant/election correlation identifiers but exclude personal secrets.

- OpenTelemetry traces and metrics: p95 latency, error rates, queue age, database connections, failed logins.
- Backup/restore drills and incident response runbook; no deployment without rollback path.

10. Testing, CI/CD and deployment

Stage	Checks
Pull request	Build, formatting, static analysis, unit tests, OpenAPI diff
Integration	Testcontainers PostgreSQL; RLS isolation; real EF migrations
Security	Cross-tenant IDOR, privilege escalation, token misuse, upload malware
Performance	1,000 concurrent-user design target, measured with representative workloads
Staging	UAT scripts, backup restore, rollback rehearsal
Production	Canary/blue-green as supported, health checks, alerting

```
dotnet new sln -n EEMS
dotnet new webapi -n EEMS.Api
dotnet ef migrations add InitialPlatform --project EEMS.Infrastructure
dotnet ef database update --project EEMS.Infrastructure
# Run with environment-specific configuration and least-privilege DB role.
```

11. System invariants

- Every tenant-owned row has tenant_id; all APIs derive tenant from authenticated context, never client input.
- An election has a type, schedule, geographic scope and lifecycle state. Position must belong to election type.
- Permissions are enforced in API and data access; hiding a menu is never authorization.
- Official results are clearly distinguished from unverified field submissions.
- Personal data is minimized; do not store inferred political preference or voter-level persuasion scores.

12. Cross-team contracts

Concern	Contract
Identifiers	UUID for tenant/election/candidate; bigint for canonical geographic codes where appropriate
Date/time	ISO 8601 UTC timestamps; election dates in local election timezone
API	/api/v1; RFC 7807 ProblemDetails; cursor/page pagination
State	Draft -> Submitted -> Reviewed -> Approved/Returned; record actor and time
Files	Object storage, malware scanning, SHA-256, signed downloads
Localization	Bangla/English display strings, stable English permission codes